Cloud Messaging Services

let the provider run the broker

Watch the 4-min video

In one line: every cloud offers managed queues, pub/sub topics, event streams and workflows, which remove broker operations and scale automatically; choose by delivery model, ordering, throughput, retention and integrations, and know each service's limits, such as message size, retention and FIFO throughput.

1 Three months of patching

Your team needs a message queue, and someone suggests running Kafka. Three months later you're patching brokers, rebalancing partitions and debugging disk alerts at night. For many teams a **managed** service would have done the job, with nothing to run.

	AWS	Azure	Google Cloud
queue	SQS (+FIFO)	Service Bus queues	Cloud Tasks / Pub/Sub
pub/sub	SNS, EventBridge	Service Bus topics, Event Grid	Pub/Sub
stream	Kinesis, MSK (Kafka)	Event Hubs (Kafka API)	Pub/Sub, Managed Kafka
workflow	Step Functions	Durable Functions, Logic Apps	Workflows

Figure 1. Rough equivalents across the three big clouds. Names change; the categories don't.

2 How to choose

- **Delivery model:** work queue (one consumer per message) or fan-out (many subscribers) or a replayable stream (lesson 24)?
- **Ordering:** SQS FIFO per message group, Service Bus sessions, Pub/Sub ordering keys, Kinesis/Event Hubs per shard or partition.
- **Throughput and size limits:** e.g. SQS messages up to 256 KB (larger via S3), FIFO throughput limits per group, shard limits in Kinesis.
- **Retention and replay:** queues keep messages days; streams keep them days to years.
- **Features:** DLQs, delays, scheduled messages, filtering, exactly-once options, Kafka-compatible APIs (MSK, Event Hubs, Confluent Cloud).
- **Cost:** per request, per throughput unit, or per hour; batch to save money.

Managed service
no brokers to patch, scales for you, pay per use; limits and features are the provider's

Self-run Kafka / RabbitMQ
full control and portability; you own upgrades, disks, rebalancing, on-call and capacity

Figure 2. Managed services trade control for far less operational work.

WORKED EXAMPLE: AZURE SERVICE BUS SESSIONS FOR PER-CUSTOMER ORDERING

An order system publishes events for many customers to one queue, setting `SessionId` to the customer ID. Workers accept sessions: Service Bus locks each session to one worker at a time, so customer A's events are processed in order, while customers B and C are handled in parallel by other workers. Failed messages are retried up to a max delivery count, then moved to the built-in dead-letter sub-queue. The same pattern on AWS is SQS FIFO with `MessageGroupId`.



one session is locked by one worker at a time → per-customer order, parallel across customers

Figure 3. Sessions give ordering per key with parallelism across keys.

COMMON MISTAKES

- Running your own broker for modest traffic with no platform team.
- Ignoring service limits until production (message size, FIFO throughput).
- Polling SQS with short polls and paying for empty receives (use long polling).
- Assuming every 'pub/sub' service keeps messages for replay.

4 Key terms

SQS / SNS: AWS queue / AWS pub-sub notifications

Kinesis / Event Hubs: managed event streams

Long polling: wait for messages instead of returning empty

EventBridge / Event Grid: event routers with filtering rules

Service Bus session: ordered, locked group of messages

5 Interview questions

Q1. When would you choose a managed messaging service over self-hosted Kafka? BASIC

When you want no broker operations, have moderate or spiky load, and the service's features and limits fit; self-host when you need full control, portability or very high throughput economics.

Q2. How do you get ordered processing in SQS? BASIC

Use a FIFO queue with MessageGroupId set to the entity; messages in a group are delivered in order, one at a time, while different groups run in parallel.

Q3. Fan out OrderPlaced to three services on AWS. SCENARIO

Publish to an SNS topic (or EventBridge) and subscribe one SQS queue per service, each with its own DLQ.

Q4. You need Kafka semantics but no Kafka operations on Azure. SCENARIO

Use Event Hubs with its Kafka-compatible endpoint, or Confluent Cloud; keep clients unchanged.

Q5. What limits catch teams out with SQS? TRICKY

256 KB message size, FIFO throughput per message group, visibility timeout shorter than processing time causing duplicates, and 14-day maximum retention.

Q6. Is EventBridge a replacement for Kafka? TRICKY

No: it's a router with rules and integrations, not a high-throughput, replayable log with consumer groups, though it has archive and replay for some use cases.

Next: SD-I-020 · Pub/Sub and Fan-Out